

JavaScript-10

Webb-API:er

30 september 2026

JavaScript – utan tjafs
Författare: Acke Strömberg



För en kanin

Webb-API:er

Webbläsaren innehåller mycket mer än bara kod för att visa HTML och CSS. Den har en mängd inbyggda verktyg och superkrafter som kallas för Web-API:er. I det här kapitlet ska vi titta närmare på två av de absolut mest användbara verktygen: hur vi sparar data direkt på användarens dator så att den finns kvar när sidan laddas om, samt hur vi skapar smart infinite scroll och lazy-loading med ett modernt verktyg som heter Intersection Observer.

Web Storage – Kom ihåg data i webbläsaren

När en användare laddar om en webbsida, eller navigerar till en ny undersida, nollställs alla JavaScript-variabler i minnet. Om du vill spara produkter i en kundvagn, eller komma ihåg om användaren har valt "mörkt tema", måste datan sparas någonstans.

För detta har vi två fantastiska verktyg:

- **LocalStorage:** Sparar datan permanent. Den finns kvar även om användaren stänger ner webbläsaren eller startar om datorn.
- **SessionStorage:** Sparar datan temporärt. Datan raderas automatiskt så fort användaren stänger den specifika fliken.

VIKTIGT !

Både **LocalStorage** och **SessionStorage** kan bara spara textsträngar. Om vi vill spara objekt eller arrayer måste vi omvandla dem till JSON-strängar först.

```
// Data vi vill spara
const installningar = {
  tema: 'morkt',
  volym: 80
};

// 1. SPARA: Vi gör om objektet till text med JSON.stringify() innan vi sparar
localStorage.setItem('anvandar_val', JSON.stringify(installningar));

// ... Sidan laddas om eller användaren stänger fliken ...

// 2. HÄMTA: Vi hämtar textsträngen från minnet
const sparadText = localStorage.getItem('anvandar_val');

// 3. OMVANDLA: Vi gör om texten till ett JavaScript-objekt igen med JSON.parse()
const laddadeInstallningar = JSON.parse(sparadText);

console.log(laddadeInstallningar.tema); // Utskrift: morkt
```

Bild 1 Med JSON.stringify och JSON.parse kan vi enkelt spara och hämta komplex data.

Intersection Observer – Styr vad som syns på skärmen

Förr i tiden, om man ville veta när ett visst element dök upp på skärmen (t.ex. för att starta en animation eller ladda fler produkter när användaren scrollat till botten), var man tvungen att lyssna på webbläsarens scroll-event. Det gjorde sidan väldigt långsam eftersom scroll-funktionen triggades hundratals gånger i sekunden.

Idag använder vi Intersection Observer API. Det är som att sätta en dold kamera på ett specifikt HTML-element på sidan. Vi ber webbläsaren att hålla utkik och säga till bara när elementet faktiskt kliver in i eller lämnar användarens synfält (skärmen).

Detta är det perfekta verktyget för att bygga Infinite Scroll (att ladda mer innehåll automatiskt när man når botten) eller Lazy Loading (att inte ladda tunga bilder förrän användaren faktiskt scrollat ner till dem).

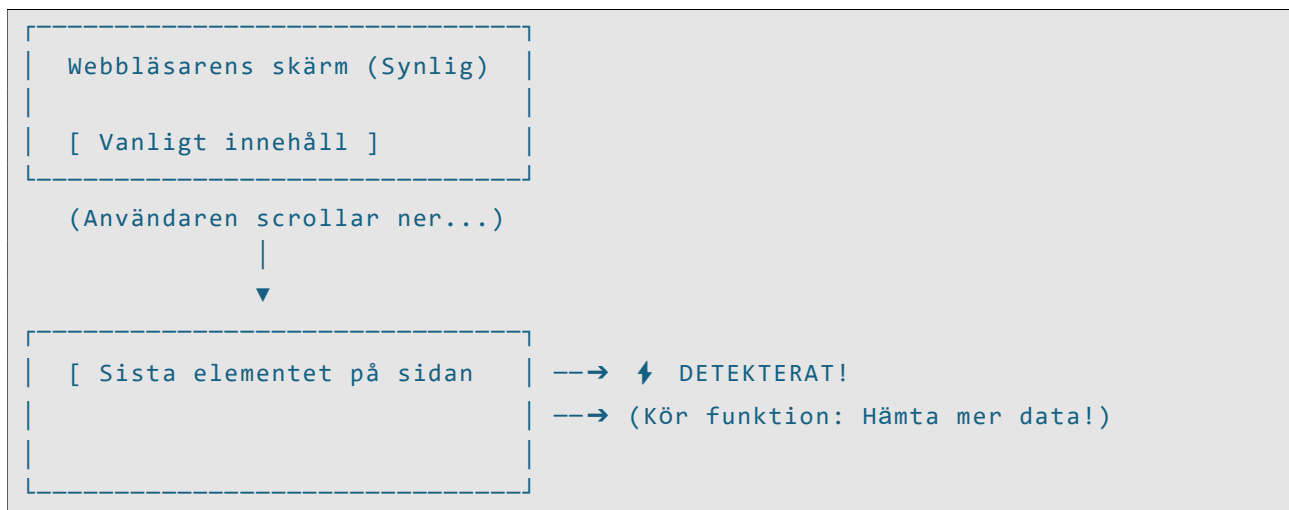


Bild 2 Observatören utlöser vår funktion när övervakade elementet blir synligt för användaren.

Här lägger vi en osynlig "laddnings-låda" längst ner på vår sida. När den syns på skärmen vet vi att det är dags att hämta mer data.

```
// 1. Funktionen som ska köras när elementet syns på skärmen
const hanteraSynlighet = (entries) => {
  entries.forEach((entry) => {
    // entry.isIntersecting blir TRUE i det ögonblick elementet syns på skärmen
    if (entry.isIntersecting) {
      console.log("Användaren har nått botten! Hämtar fler produkter...");
      // Här skulle du göra ett fetch-anrop för att hämta mer data!
    }
  });
};

// 2. Skapa själva observatören (kameran)
const observator = new IntersectionObserver(hanteraSynlighet);

// 3. Säg åt kameran vilket element den ska hålla koll på
const laddningsMarkor = document.querySelector('#bottom-marker');
observator.observe(laddningsMarkor);
```

Bild 3 Genom att använda IntersectionObserver reagerar koden bara när det behövs.

Lazy loading

Intersection Observer kan användas för att skapa en osynlig markör längst ner på en sida för att bygga en "infinite scroll". Men det här API:et har en annan ovärderlig superkraft: **Lazy Loading**. I det här kapitlet ska vi fördjupa oss i hur vi sparar enormt mycket nätverkstrafik genom att inte ladda bilder förrän de faktiskt syns på skärmen, samt lära oss hur vi skapar våra helt egna händelser i webbläsaren med **Custom Events**.

Avancerad Intersection Observer: Lazy Loading av bilder

När en webbläsare möter en `` i din HTML, laddar den ner bilden direkt, även om bilden ligger längst ner på sidan och användaren kanske aldrig scollar dit. Om du har en blogg eller webbshop med 50 högupplösta bilder blir sidan seg och slösar på användarens mobildata.

Med **Lazy Loading** lägger vi bildens riktiga länk i ett tillfälligt attribut (ofta kallat `data-src`) och lämnar den vanliga `src` tom (eller med en liten grå placeholder-ruta). Sedan ber vi vår *Intersection Observer* att hålla koll på bilderna. Så fort en bild kliver in på skärmen flyttar vi länken till `src` så att bilden laddas!

```
UTANFÖR SKÄRMEN: 
(Ingen tung nedladdning!)
|
▼ (Användaren scollar ner så att bilden syns)
PÅ SKÄRMEN (⚡): 
(Bilden laddas precis i rätt tid!)
```

Bild 4 Bilden laddas först när observatören upptäcker att den kliver in i webbläsarens synfält.

```
// 1. Skapa funktionen som körs när en bild blir synlig
const laddaBildBlixtsnabbt = (entries, observer) => {
  entries.forEach(entry => {
    // Om bilden syns på skärmen
    if (entry.isIntersecting) {
      const bild = entry.target;

      // Flytta den riktiga länken från data-src till src
      bild.src = bild.dataset.src;

      console.log(`⚡ Laddade precis in bilden: ${bild.src}`);

      // Sluta övervaka den här bilden (den är ju redan laddad!)
      observer.unobserve(bild);
    }
  });
};

// 2. Starta observatören
const bildObservator = new IntersectionObserver(laddaBildBlixtsnabbt);

// 3. Hitta alla bilder på sidan som ska laddas långsamt
const doldaBilder = document.querySelectorAll('img[data-src]');
doldaBilder.forEach(bild => bildObservator.observe(bild));
```

Bild 5 Genom att köra `unobserve()` stänger vi av övervakningen för varje enskild bild så fort den har laddats, vilket sparar datorns processorkraft.

Custom Events – Skapa dina egna händelser

Du är van vid att lyssna på inbyggda händelser som click, submit och input. Men i stora JavaScript-applikationer vill man ibland att olika moduler ska kunna prata med varandra utan att sitta ihop hårt i koden. Då kan vi skapa helt egna händelser med **CustomEvent**.

Tänk dig att du har en modul för en kundvagn och en helt annan modul för en toppmeny. När en användare klickar på "Köp" inuti kundvagnen, vill du att toppmenyn ska uppdatera sin lilla siffra över hur många artiklar som finns i vagnen.

Istället för att kundvagnsmodulen ska gå in och ändra i toppmenyns HTML-kod, kan kundvagnen bara skicka ut ett eget event: "kundvagnUppdaterad". Toppmenyn ligger sedan och lyssnar på detta event och sköter sig själv!

```
STEIG 1: [ Kundvagn-Modul ]
      |
      | (Skickar ut: "vagnÄndrad" med data)
      ▼
STEIG 2: [ Webbbläsfönstret (window) ]
      |
      | (Händelsen ropas ut globalt)
      ▼
STEIG 3: [ Toppmeny-Modul ] → ⚡ Fångar händelsen & uppdaterar siffran!
```

Bild 3: Custom Events låter delar av din applikation att kommunicera oberoende av varandra.

Här är koden för hur du skapar, skickar iväg (dispatchar) och lyssnar på en helt egen händelse:

```
// MODUL 1: Kundvagnen (Här skapas och skickas händelsen)
function läggTillIKundvagn() {
  console.log("En vara lades till i vagnen!");

  // Skapa ett eget event. I objektet 'detail' kan vi skicka med valfri data!
  const vagnEvent = new CustomEvent('vagnÄndrad', {
    detail: { antalArtiklar: 4, totalsumma: 899 }
  });

  // Skicka ut eventet i hela webbläsarfönstret så alla kan höra det
  window.dispatchEvent(vagnEvent);
}

// MODUL 2: Toppmenyn (Här lyssnar vi på händelsen helt fristående)
window.addEventListener('vagnÄndrad', (event) => {
  // Vi kommer åt vår medskickade data via event.detail
  const info = event.detail;

  const menySiffra = document.querySelector('#meny-antal');
  menySiffra.textContent = info.antalArtiklar;

  console.log(`Toppmenyn hörde. Visar nu ${info.antalArtiklar} prylar.`);
});

// Testkör (Simulera ett köp efter 2 sekunder)
setTimeout(läggTillIKundvagn, 2000);
```

Bild 4: Med event.detail kan vi skicka med komplexa JS-objekt och arrayer mellan helt olika filer och moduler i vårt projekt.